

RELATÓRIO DE AUDITORIA DE SEGURANÇA

Insulog

API api-insulog + aplicativo Flutter

17/09/2026 • Revisão de código e validação local

API inteira no clone insulog-back-clone; frontend irmão insulog-mobile-front-clone em leitura complementar para permissões, segredos, saída e transporte. Código próprio, configurações, CI, documentação e histórico alcançável local; bundle web local inventariado. Sem acesso ao MySQL real, infraestrutura de produção ou exploração remota.

Nota metodológica

As cinco categorias foram adaptadas a Express/MySQL e Flutter: isolamento por dono nas queries, autorização no servidor, referências diretas em todas as rotas, segredos no código/histórico/bundle e sinks de HTML/JavaScript. Achados adicionais de credenciais foram identificados à parte. Só riscos sustentados no código são reportados; ausência de contexto de produção é explicitada.

API: feb285d76d9b059e5e2f440c55472a95abbd06ca

App: ee623bbf1b0a538e140141388dab23982705677b

10 achados • 49 registros de rota • 36 commits locais

Resumo executivo

A API expõe dados privados e aceita alterações por IDs controlados pelo chamador. O login confere senha, mas não estabelece uma identidade exigida pelas demais rotas. As prioridades são autenticação com posse, rotação das credenciais históricas e proteção das senhas em trânsito e logs.

Crítica	Alta	Média	Baixa	Informativa
2	6	2	0	0



- crítica: 2
- alta: 6
- média: 2
- baixa: 0
- informativa: 0

Achados por categoria

1. Isolamento		1
2. Permissões		2
3. IDOR		3
4. Chaves expostas		1
5. XSS		0
Adicional		3

Sete achados no escopo principal e três adicionais. Contagem por achado, sem multiplicar pelo número de rotas. Ausência de XSS verificado não significa aprovação geral de segurança.

Stack, escopo e método

JavaScript CommonJS / Node.js (CI 24), Express 5.2.1; mysql2 com SQL manual, sem ORM/query builder; login por senha script (fallback legado), sem token/sessão de autorização. Frontend Dart/Flutter, http e SharedPreferences. PDFKit/ExcelJS geram relatórios. CI GitHub Actions para Node/Flutter; nenhum Docker, Compose, Helm ou Terraform próprio encontrado.

1. Isolamento

MySQL com SQL manual e filtros `id_usuario`; sem RLS, middleware de tenant ou identidade autenticada no código. Não há entidade organização/workspace. Relação `paciente.id_medico` existe, mas não é usada para autorizar consultas.

A01: listagens sem filtro. Filtros existentes aceitam ID do cliente; IDs específicos agrupados na categoria 3 para não contar duas vezes.

2. Permissões

Flutter/Dart, não navegador como fronteira de confiança. Busca de gates `isAdmin/canEdit/role/tipo_usuario` e rastreamento de APIs.

Nenhum gate visual por papel encontrado para cruzar 1:1. Cadastro fixa paciente no cliente; servidor aceita medico (A06). Escrita dos catálogos está aberta (A05). Não há privilégio admin implementado demonstrado.

3. IDOR

Todos os 49 registros de rota Express foram rastreados por controller, serviço e repository; 48 são alcançáveis.

A02/A03/A04. GET `/usuarios/:tipo_usuario` é sombreado por `/:id`; não foi tratado como endpoint explorável separado. Catálogos são globais; suas escritas pertencem à categoria 2.

4. Chaves expostas

Código/config/CI/documentação, defaults, todas as refs Git locais e artefatos locais do frontend.

A07: duas credenciais históricas. HEAD usa `process.env`, sem defaults secretos. Startup não valida configuração e captura falha de conexão, mas não há default secreto comprovado; isso não foi contado como outro achado. Não há Docker/Compose/Helm/Terraform próprios nos arquivos versionados.

5. XSS

Flutter Text/TextSpan, assets SVG locais e bootstrap web; Express JSON, PDFKit e ExcelJS. Busca de sinks HTML, URLs, eval, Markdown e templates.

Nenhum fluxo XSS verificado. Sinks de HTML/e-mail não se aplicam aos caminhos implementados. Não há biblioteca dedicada a sanitizar HTML nas dependências diretas; sua ausência não é vulnerabilidade onde HTML não é interpretado.

Adicional

Achados diretamente observados de proteção de credenciais, fora das cinco categorias.

A08: logs; A09: persistência local; A10: transporte HTTP. Não são classificados como hardcoded nem XSS.

Pontos fortes e pontos fracos

Hash de senha nas novas gravações

Cadastro e atualização geram script com salt aleatório de 16 bytes; comparação de hashes usa `timingSafeEqual`. Ressalva: permanece fallback para senhas legadas em texto puro e A01 expõe a coluna senha.

[src/services/passwordService.js:6-9,27-34,42-47](#) | [src/services/userService.js:70-74,87-91](#)

SQL parametrizado nos valores

Os oito repositories usam `execute` e placeholders para IDs e valores. `LIMIT` é numérico validado nos serviços; direção de `ORDER BY` é escolhida em lista fixa. Isso reduz injeção SQL, mas não prova autorização.

[src/repositories/userRepository.js:42-66](#) | [src/services/registroGlicoseService.js:22-35](#) | [src/services/registroInsulinaService.js:20-33](#) | [src/repositories/registroGlicoseRepository.js:301-310](#)

Saída sem interpretação de HTML nos caminhos revisados

Flutter usa `Text/TextSpan`; PDFKit usa `doc.text` para observações; ExcelJS recebe strings. Não há e-mail/template HTML, `WebView`, interpretador Markdown/HTML ou eval alimentado por usuário no código próprio revisado. O SVG do PDF é asset local fixo.

[../insulog-mobile-front-clone/lib/widgets/home/home_header_widget.dart:24-25](#) | [src/reports/pdfReport.js:6-7,54-60,314-318](#) | [src/reports/xlsxReport.js:54-63](#)

Validação e cabeçalhos na exportação

ID inteiro seguro, datas reais e ordenadas, formatos limitados; no-store, attachment e nosniff. O cliente verifica MIME/assinatura e restringe nome de arquivo. São proteções válidas, porém não substituem a posse ausente em A04.

[src/services/exportacaoService.js:74-112](#) | [src/controllers/exportacaoController.js:51-67](#) | [../insulog-mobile-front-clone/lib/services/api/report_export_service.dart:86-115](#)

Credenciais externas no HEAD e CI restrito da API

Configuração MySQL atual usa apenas variáveis de ambiente, sem senha pública de fallback. Workflow da API declara contents: read. Ausência de segredo no HEAD não elimina o achado histórico A07.

[src/config/database.js:8-13](#) | [.github/workflows/api-tests.yml:11-12](#)

Redação de credenciais no logger do cliente

O cliente mascara recursivamente campos com senha/password/token. É um controle parcial de logs, não sanitização de HTML; as respostas clínicas ainda são logadas pelo cliente e o backend não aplica essa proteção.

[../insulog-mobile-front-clone/lib/services/api/api_service.dart:244-254,274-291](#)

Pontos fracos centrais

Não há router de dados privados com posse integralmente protegida. As validações encontradas verificam formato ou existência. Os filtros `id_usuario` não isolam usuários quando o próprio solicitante escolhe esse ID. CORS não é autenticação. Credenciais históricas e senhas em logs/armazenamento/tráfego agravam a exposição.

Achados por categoria

1. Isolamento

Severidade	Arquivo:linha (entrada)	Descrição
ALTA	src/controllers/userController.js:3-6	A01 — Listagens globais expõem dados e hashes de senha

2. Permissões

Severidade	Arquivo:linha (entrada)	Descrição
ALTA	src/routes/periodoRoutes.js:8-10	A05 — Catálogos compartilhados aceitam escrita anônima
MÉDIA	../insulog-mobile-front-clone/lib/services/api/auth_service.dart:89-95	A06 — Tipo de usuário é aceito do cliente sem aprovação

3. IDOR

Severidade	Arquivo:linha (entrada)	Descrição
CRÍTICA	src/routes/userRoutes.js:8-14	A02 — Conta de terceiros pode ser alterada, inclusive a senha
CRÍTICA	src/controllers/registroGlicoseController.js:44-48	A03 — Registros clínicos e objetos vinculados não validam posse
ALTA	src/routes/exportacaoRoutes.js:6-9	A04 — Relatório clínico pode ser exportado para qualquer paciente

4. Chaves expostas

Severidade	Arquivo:linha (entrada)	Descrição
ALTA	.env:1-5 @ 147015e9442d	A07 — Credenciais MySQL permanecem recuperáveis no Git

5. XSS

Nenhum achado verificado. Nenhum fluxo XSS verificado. Sinks de HTML/e-mail não se aplicam aos caminhos implementados. Não há biblioteca dedicada a sanitizar HTML nas dependências diretas; sua ausência não é vulnerabilidade onde HTML não é interpretado.

Adicional

Severidade	Arquivo:linha (entrada)	Descrição
ALTA	src/app.js:39-48	A08 — Senhas e respostas sensíveis são gravadas nos logs

Achados por categoria

Severidade	Arquivo:linha (entrada)	Descrição
MÉDIA	../insulog-mobile-front-clone/lib/services/local/saved_login_service.dart:20-28	A09 — App persiste senha em preferências locais
ALTA	../insulog-mobile-front-clone/lib/services/local/api_ip_service.dart:18-26	A10 — App envia login e dados clínicos por HTTP

A01 · Evidências e análise

ALTA

Listagens globais expõem dados e hashes de senha

1. Isolamento

As seis listagens de dados privados não recebem identidade autenticada e executam consultas sem filtro de dono. GET /usuarios devolve SELECT * diretamente, incluindo a coluna senha. Os IDs expostos também facilitam as referências diretas dos demais achados.

Impacto

Leitura em massa de contas, hashes bcrypt, medições, doses, alarmes, preferências e metadados de exportação. Se houver contas legadas em texto puro, a senha também é retornada; a existência delas em produção não foi aferida.

Condições de explorabilidade

Acesso de rede à API e linhas existentes nas tabelas; não depende de login ou feature flag. Hashes não equivalem automaticamente a senhas recuperadas.

Arquivo por arquivo, linha por linha

[src/controllers/userController.js:3-6](#)

```
3: async function index(req, res, next) {  
4:   try {  
5:     const users = await userService.listUsers()  
6:     return res.status(200).json(users)
```

[src/repositories/userRepository.js:4-9](#)

```
4: async function findAll() {  
5:   const [rows] = await db.execute(  
6:     'SELECT * FROM usuario ORDER BY id_usuario ASC'  
7:   )  
8:  
9:   return rows
```

[src/repositories/registroGlicoseRepository.js:5-9](#)

```
5:   const [rows] = await db.execute(  
6:     `SELECT rg.id_registro, rg.id_usuario, rg.nivel_glicose, rg.data_hora, p.descricao AS periodo  
7:     FROM registroglicose rg  
8:     LEFT JOIN periodo p ON p.id_periodo = rg.id_periodo  
9:     ORDER BY rg.id_registro ASC`
```

[src/repositories/registroInsulinaRepository.js:4-7](#)

```
4: async function findAll() {  
5:   const [rows] = await db.execute(  
6:     'SELECT id_registro_insulina, id_registro, id_tipo_insulina, unidade_insulina FROM registroinsul  
   ina ORDER BY id_registro_insulina ASC'  
7:   )
```

A01 · Evidências e análise · continuação

src/repositories/alarmeRepository.js:14-17

```
14: async function findAll() {
15:   const [rows] = await db.execute(
16:     'SELECT id_alarme, id_usuario, data_hora, id_periodo, id_registro, dias_semana, ativo, tem_som,
      tem_vibricao FROM alarme ORDER BY id_alarme ASC'
17:   )
```

src/repositories/configuracaoRepository.js:4-7

```
4: async function findAll() {
5:   const [rows] = await db.execute(
6:     'SELECT id_configuracao, id_usuario, idioma, tema, notificacoes FROM configuracao ORDER BY id_co
      nfiguracao ASC'
7:   )
```

src/repositories/exportacaoRepository.js:4-7

```
4: async function findAll() {
5:   const [rows] = await db.execute(
6:     'SELECT id_exportacao, id_usuario, data, descricao FROM exportacao ORDER BY id_exportacao ASC'
7:   )
```

src/services/passwordService.js:42-47

```
42:   if (isHashedPassword(storedPassword)) {
43:     return verifyHashedPassword(password, storedPassword)
44:   }
45:
46:   // Compatibilidade temporaria com usuarios antigos salvos em texto puro.
47:   return storedPassword === password
```

Correção recomendada

Exigir identidade validada, filtrar cada coleção pelo dono autorizado ou vínculo médico-paciente verificado e retornar DTOs com lista explícita de campos. Nunca serializar senha/hash. Migrar senhas legadas caso existam.

A02 · Evidências e análise

CRÍTICA

Conta de terceiros pode ser alterada, inclusive a senha

3. IDOR

GET, PUT e DELETE /usuarios/:id confiam apenas no ID. A atualização aceita uma nova senha, gera o hash e atualiza a conta sem sessão, senha atual ou fluxo de recuperação. A reprodução local mudou a senha de uma conta fictícia e realizou login com a nova senha.

Impacto

Tomada de conta e alteração de nome, e-mail e tipo de usuário; leitura de perfil. Exclusão também alcança SQL sem autorização, podendo depender das foreign keys e dos registros vinculados.

Condições de explorabilidade

API acessível e ID existente. Para atualização, enviar os campos obrigatórios; nenhuma credencial prévia é exigida. DELETE pode ser impedido por integridade referencial, sem corrigir a falha de autorização.

Arquivo por arquivo, linha por linha

src/routes/userRoutes.js:8-14

```
8: router.get('/', userController.index);
9: router.get('/:id', userController.show);
10: router.get('/:tipo_usuario', userController.showByType);
11:
12: router.post('/', userController.create);
13: router.delete('/:id', userController.deleteById);
14: router.put('/:id', userController.update);
```

src/controllers/userController.js:53-57

```
53: async function update(req, res, next) {
54:   try {
55:     const { id } = req.params
56:     const user = await userService.updateUser(id, req.body)
57:     return res.status(200).json(user)
```

src/services/userService.js:78-91

```
78: async function updateUser(id, data) {
79:   const { nome, email, senha, tipo_login, tipo_usuario, id_medico, crm } = data
80:
81:   if (!nome || !email || !senha || !tipo_login || !tipo_usuario) {
82:     const error = new Error('Todos os campos sao obrigatorios')
83:     error.statusCode = 400
84:     throw error
85:   }
86:
87:   const senhaCriptografada = passwordService.hashPassword(senha)
88:
89:   return await userRepository.update(
90:     id,
91:     { nome, email, senha: senhaCriptografada, tipo_login, tipo_usuario, id_medico, crm }
```

A02 · Evidências e análise · continuação

[src/repositories/userRepository.js:42-45](#)

```
42: async function findById(id) {
43:   const [rows] = await db.execute(
44:     'SELECT id_usuario, nome, email, tipo_login, tipo_usuario FROM usuario WHERE id_usuario = ?',
45:     [id]
```

[src/repositories/userRepository.js:134-136](#)

```
134:   await conn.execute(
135:     'UPDATE usuario SET nome = ?, email = ?, senha = ?, tipo_login = ?, tipo_usuario = ? WHERE i
    d_usuario = ?',
136:     [nome, email, senha, tipo_login, tipo_usuario, id]
```

[src/repositories/userRepository.js:18-30](#)

```
18:   await conn.execute(
19:     'DELETE FROM paciente WHERE id_usuario = ?',
20:     [id]
21:   )
22:
23:   await conn.execute(
24:     'DELETE FROM medico WHERE id_usuario = ?',
25:     [id]
26:   )
27:
28:   await conn.execute(
29:     'DELETE FROM usuario WHERE id_usuario = ?',
30:     [id]
```

Correção recomendada

Autenticar as rotas, vincular o perfil à identidade do servidor e exigir autorização específica para terceiros. Separar troca de senha de edição comum, com reautenticação ou token de recuperação de uso único; limitar exclusão.

A03 · Evidências e análise

CRÍTICA

Registros clínicos e objetos vinculados não validam posse

3. IDOR

Os handlers de glicose, insulina, alarmes, configurações e metadados de exportação usam IDs de path/body/query sem comparar com dono autenticado. Verificar existência com findById não valida posse. POST aceita id_usuario ou id_registro de terceiros; PUT pode mudar esses vínculos. Dashboard e histórico também aceitam o paciente escolhido pelo chamador.

Impacto

Leitura, criação, adulteração e exclusão de medições e doses de terceiros; alteração de alarmes e preferências. O impacto crítico decorre da integridade dos registros clínicos, não de um comando demonstrado a dispositivo de administração de insulina.

Condições de explorabilidade

API alcançável e IDs/referências válidos. Constraints podem rejeitar IDs inexistentes, mas não isolam donos existentes. Sem feature flag ou sessão necessária.

Arquivo por arquivo, linha por linha

src/controllers/registroGlicoseController.js:44-48

```
44: async function getDashboard(req, res, next) {
45:   try {
46:     const { id_usuario, dataInicio, dataFim } = req.query
47:     const dashboardDados = await registroGlicoseService.getDashboardDados(id_usuario, dataInicio, dataFim)
48:     return res.status(200).json(dashboardDados)
```

src/services/registroGlicoseService.js:125-130

```
125: function montarRegistroCompleto(data, registroAtual = {}) {
126:   const id_usuario = data.id_usuario ?? registroAtual.id_usuario
127:   const nivel_glicose = data.nivel_glicose ?? registroAtual.nivel_glicose
128:   const id_periodo = data.id_periodo ?? registroAtual.id_periodo
129:   const data_hora = data.data_hora ?? registroAtual.data_hora ?? formatarDataHoraAtual()
130:   const observacao = data.observacao ?? registroAtual.observacao ?? null
```

src/services/registroGlicoseService.js:356-365

```
356: async function updateRegistroGlicose(id, data) {
357:   const registroGlicose = await registroGlicoseRepository.findById(id)
358:
359:   if (!registroGlicose) {
360:     const error = new Error('Registro de glicose nao encontrado')
361:     error.statusCode = 404
362:     throw error
363:   }
364:
365:   const glicose = montarRegistroCompleto(data, registroGlicose)
```

A03 · Evidências e análise · continuação

src/repositories/registroGlicoseRepository.js:51-54

```
51:     LEFT JOIN alarme a ON a.id_registro = rg.id_registro
52:     LEFT JOIN periodo pl ON pl.id_periodo = a.id_periodo
53:     WHERE rg.id_registro = ?`,
54:     [id]
```

src/repositories/registroGlicoseRepository.js:197-205

```
197:     await conn.execute(
198:       'UPDATE registroglicose SET id_usuario = ?, nivel_glicose = ?, data_hora = ?, id_periodo = ?
    , observacao = ? WHERE id_registro = ?',
199:       [
200:         glicose.id_usuario,
201:         glicose.nivel_glicose,
202:         glicose.data_hora,
203:         glicose.id_periodo,
204:         glicose.observacao ?? null,
205:         id
```

src/repositories/registroGlicoseRepository.js:277-290

```
277:     await conn.execute(
278:       'DELETE FROM alarme WHERE id_registro = ?',
279:       [id]
280:     )
281:
282:     await conn.execute(
283:       'DELETE FROM registroinsulina WHERE id_registro = ?',
284:       [id]
285:     )
286:
287:     await conn.execute(
288:       'DELETE FROM registroglicose WHERE id_registro = ?',
289:       [id]
290:     )
```

src/repositories/registroGlicoseRepository.js:304-310

```
304:     const [rows] = await db.execute(
305:       `SELECT rg.id_registro, rg.id_usuario, rg.nivel_glicose, rg.data_hora, p.descricao AS periodo
306:       FROM registroglicose rg
307:       LEFT JOIN periodo p ON p.id_periodo = rg.id_periodo
308:       WHERE rg.id_usuario = ? AND rg.data_hora BETWEEN ? AND ?
309:       ORDER BY rg.data_hora ${ordemData}`,
310:       [id_usuario, dataInicio, dataFim]
```

src/repositories/registroInsulinaRepository.js:12-15

```
12: async function findById(id) {
13:   const [rows] = await db.execute(
14:     'SELECT id_registro_insulina, id_registro, id_tipo_insulina, unidade_insulina FROM registroinsu
    lina WHERE id_registro_insulina = ?',
15:     [id]
```

A03 · Evidências e análise · continuação

src/repositories/registroInsulinaRepository.js:51-54

```
51:     const [result] = await conn.execute(  
52:       'INSERT INTO registroinsulina (id_registro, id_tipo_insulina, unidade_insulina) VALUES (?, ?,  
    ?)',  
53:       [id_registro, id_tipo_insulina, unidade_insulina]  
54:     )
```

src/repositories/registroInsulinaRepository.js:79-81

```
79:     await conn.execute(  
80:       'UPDATE registroinsulina SET id_registro = ?, id_tipo_insulina = ?, unidade_insulina = ? WHERE  
    E id_registro_insulina = ?',  
81:       [id_registro, id_tipo_insulina, unidade_insulina, id]
```

src/repositories/registroInsulinaRepository.js:100-104

```
100: async function deleteById(id) {  
101:   await db.execute(  
102:     'DELETE FROM registroinsulina WHERE id_registro_insulina = ?',  
103:     [id]  
104:   )
```

src/repositories/alarmeRepository.js:22-25

```
22: async function findByUsuarioId(usuarioId) {  
23:   const [rows] = await db.execute(  
24:     'SELECT id_alarme, id_usuario, data_hora, id_periodo, id_registro, dias_semana, ativo, tem_som,  
    tem_vibracao FROM alarme WHERE id_usuario = ? ORDER BY data_hora ASC',  
25:     [usuarioId]
```

src/repositories/alarmeRepository.js:30-33

```
30: async function findById(id) {  
31:   const [rows] = await db.execute(  
32:     'SELECT id_alarme, id_usuario, data_hora, id_periodo, id_registro, dias_semana, ativo, tem_som,  
    tem_vibracao FROM alarme WHERE id_alarme = ?',  
33:     [id]
```

src/repositories/alarmeRepository.js:46-48

```
46:     const [result] = await conn.execute(  
47:       'INSERT INTO alarme (id_usuario, data_hora, id_periodo, id_registro, dias_semana, ativo, tem_  
    som, tem_vibracao) VALUES (?, ?, ?, ?, ?, ?, ?, ?)',  
48:       [id_usuario, data_hora, id_periodo || null, id_registro || null, dias_semana.join(','), ativo  
    , tem_som, tem_vibracao]
```

src/repositories/alarmeRepository.js:79-81

```
79:     await conn.execute(  
80:       'UPDATE alarme SET id_usuario = ?, data_hora = ?, id_periodo = ?, id_registro = ?, dias_seman  
    a = ?, ativo = ?, tem_som = ?, tem_vibracao = ? WHERE id_alarme = ?',  
81:       [id_usuario, data_hora, id_periodo || null, id_registro || null, dias_semana.join(','), ativo  
    , tem_som, tem_vibracao, id]
```

A03 · Evidências e análise · continuação

src/repositories/alarmeRepository.js:105-108

```
105: async function deleteById(id) {  
106:   await db.execute(  
107:     'DELETE FROM alarme WHERE id_alarme = ?',  
108:     [id]
```

src/repositories/configuracaoRepository.js:12-15

```
12: async function findById(id) {  
13:   const [rows] = await db.execute(  
14:     'SELECT id_configuracao, id_usuario, idioma, tema, notificacoes FROM configuracao WHERE id_conf  
    iguracao = ?',  
15:     [id]
```

src/repositories/configuracaoRepository.js:28-30

```
28:   const [result] = await conn.execute(  
29:     'INSERT INTO configuracao (id_usuario, idioma, tema, notificacoes) VALUES (?, ?, ?, ?)',  
30:     [id_usuario, idioma, tema, notificacoes]
```

src/repositories/configuracaoRepository.js:57-59

```
57:   await conn.execute(  
58:     'UPDATE configuracao SET id_usuario = ?, idioma = ?, tema = ?, notificacoes = ? WHERE id_conf  
    iguracao = ?',  
59:     [id_usuario, idioma, tema, notificacoes, id]
```

src/repositories/configuracaoRepository.js:79-82

```
79: async function deleteById(id) {  
80:   await db.execute(  
81:     'DELETE FROM configuracao WHERE id_configuracao = ?',  
82:     [id]
```

src/repositories/exportacaoRepository.js:12-15

```
12: async function findById(id) {  
13:   const [rows] = await db.execute(  
14:     'SELECT id_exportacao, id_usuario, data, descricao FROM exportacao WHERE id_exportacao = ?',  
15:     [id]
```

src/repositories/exportacaoRepository.js:28-30

```
28:   const [result] = await conn.execute(  
29:     'INSERT INTO exportacao (id_usuario, data, descricao) VALUES (?, ?, ?)',  
30:     [id_usuario, data, descricao]
```

src/repositories/exportacaoRepository.js:56-58

```
56:   await conn.execute(  
57:     'UPDATE exportacao SET id_usuario = ?, data = ?, descricao = ? WHERE id_exportacao = ?',  
58:     [id_usuario, data, descricao, id]
```

src/repositories/exportacaoRepository.js:77-80

```
77: async function deleteById(id) {  
78:   await db.execute(  
79:     'DELETE FROM exportacao WHERE id_exportacao = ?',  
80:     [id]
```

A03 · Evidências e análise · continuação

Correção recomendada

Exigir sessão e aplicar predicados de dono em leitura e escrita. Para insulina, validar o dono pela junção com registroglicose. Derivar id_usuario da sessão, verificar referências cruzadas e impedir transferência arbitrária de dono; manter verificação e mutação na mesma transação quando necessário.

A04 · Evidências e análise

ALTA

Relatório clínico pode ser exportado para qualquer paciente

3. IDOR

GET /exportacoes/relatorio recebe id_usuario da query e repassa ao repository. A query filtra esse ID, mas nenhuma etapa verifica a identidade do solicitante ou o vínculo com o paciente. A validação de formato/data não é autorização.

Impacto

Extração de identidade e histórico clínico consolidado em PDF/XLSX de terceiros, com observações e aplicações de insulina.

Condições de explorabilidade

ID existente, intervalo válido e registros no período. Sem token/cookie necessário; download de PDF confirmado com banco simulado. Não foi acessado banco real.

Arquivo por arquivo, linha por linha

src/routes/exportacaoRoutes.js:6-9

```
6: router.get('/', exportacaoController.index)
7: router.get('/relatorio', exportacaoController.gerarRelatorio)
8: router.get('/:id', exportacaoController.show)
9: router.post('/', exportacaoController.create)
```

src/controllers/exportacaoController.js:54-60

```
54:   const { id_usuario, dataInicio, dataFim, formato } = req.query
55:   const arquivo = await exportacaoService.gerarRelatorio({
56:     idUsuario: id_usuario,
57:     dataInicio,
58:     dataFim,
59:     formato
60:   })
```

src/services/exportacaoService.js:114-118

```
114:   const dados = await exportacaoRepository.findDadosRelatorio(
115:     Number(idUsuario),
116:     `${dataInicio} 00:00:00`,
117:     dataFim
118:   )
```

src/repositories/exportacaoRepository.js:84-87

```
84: async function findDadosRelatorio(idUsuario, dataInicio, dataFim) {
85:   const [usuarios] = await db.execute(
86:     'SELECT id_usuario, nome, email FROM usuario WHERE id_usuario = ?',
87:     [idUsuario]
```

src/repositories/exportacaoRepository.js:104-111

```
104:   FROM registroglicose rg
105:   LEFT JOIN periodo p ON p.id_periodo = rg.id_periodo
106:   LEFT JOIN registroinsulina ri ON ri.id_registro = rg.id_registro
107:   LEFT JOIN tipoinsulina ti ON ti.id_tipo_insulina = ri.id_tipo_insulina
```


A04 · Evidências e análise · continuação

```
108: WHERE rg.id_usuario = ? AND rg.data_hora >= ?
109:     AND rg.data_hora < DATE_ADD(?, INTERVAL 1 DAY)
110: ORDER BY rg.data_hora ASC, ri.id_registro_insulina ASC`,
111: [idUserio, dataInicio, dataFim]
```

[../insulog-mobile-front-clone/lib/services/api/report_export_service.dart:58-70](#)

```
58:     final uri = Uri.parse('${await _baseUrl()}/exportacoes/relatorio')
59:     .replace(
60:       queryParameters: {
61:         'id_usuario': '$userId',
62:         'dataInicio': firstDay,
63:         'dataFim': lastDay,
64:         'formato': format.name,
65:       },
66:     );
67:     final get = _client?.get ?? http.get;
68:     final response = await get(
69:       uri,
70:       headers: {'Accept': format.mimeType},
```

Correção recomendada

Autenticar antes de gerar o relatório, usar o próprio ID da sessão ou validar explicitamente o vínculo com outro paciente. Reutilizar a mesma política para os dois formatos.

A05 · Evidências e análise

ALTA

Catálogos compartilhados aceitam escrita anônima

2. Permissões

POST, PUT e DELETE de períodos e tipos de insulina não verificam privilégio. São catálogos globais sem dono individual; portanto a falha é de autorização funcional, e não IDOR de tenant. Não há gate de papel correspondente no frontend: os endpoints podem ser chamados diretamente.

Impacto

Qualquer chamador pode inserir/alterar rótulos compartilhados usados nos registros e relatórios. Exclusões de entradas vinculadas podem ser bloqueadas por foreign keys; inserção e edição continuam sem autorização.

Condições de explorabilidade

API acessível; campos descricao/nome preenchidos. PUT/DELETE exigem ID existente. Não foi presumido um papel admin que não existe no projeto.

Arquivo por arquivo, linha por linha

[src/routes/periodoRoutes.js:8-10](#)

```
8: router.post('/', periodoController.create)
9: router.put('/:id', periodoController.update)
10: router.delete('/:id', periodoController.deleteById)
```

[src/routes/typeInsuRoutes.js:8-10](#)

```
8: router.post('/', typeInsuController.create)
9: router.put('/:id', typeInsuController.update)
10: router.delete('/:id', typeInsuController.deleteById)
```

[src/services/periodoService.js:40-46](#)

```
40: if (!data.descricao) {
41:   const error = new Error('0 campo descricao é obrigatório')
42:   error.statusCode = 400
43:   throw error
44: }
45:
46: return await periodoRepository.update(id, { descricao: data.descricao })
```

[src/repositories/periodoRepository.js:28-30](#)

```
28: const [result] = await conn.execute(
29:   'INSERT INTO periodo (descricao) VALUES (?)',
30:   [descricao]
```

[src/repositories/periodoRepository.js:54-56](#)

```
54: await conn.execute(
55:   'UPDATE periodo SET descricao = ? WHERE id_periodo = ?',
56:   [descricao, id]
```

A05 · Evidências e análise · continuação

[src/repositories/periodoRepository.js:73-76](#)

```
73: async function deleteById(id) {  
74:   await db.execute(  
75:     'DELETE FROM periodo WHERE id_periodo = ?',  
76:     [id]
```

[src/repositories/typeInsuRepository.js:21-24](#)

```
21: async function deleteById(id) {  
22:   await db.execute(  
23:     'DELETE FROM tipoinsulina WHERE id_tipo_insulina = ?',  
24:     [id]
```

[src/repositories/typeInsuRepository.js:36-38](#)

```
36:   await conn.execute(  
37:     'UPDATE tipoinsulina SET nome = ? WHERE id_tipo_insulina = ?',  
38:     [nome, id]
```

[src/repositories/typeInsuRepository.js:63-65](#)

```
63:   const [result] = await conn.execute(  
64:     'INSERT INTO tipoinsulina (nome) VALUES (?)',  
65:     [nome]
```

Correção recomendada

Definir política de manutenção dos catálogos e aplicá-la no servidor às seis operações de escrita. Se não forem gerenciáveis pela API, remover as rotas de mutação.

A06 · Evidências e análise

MÉDIA

Tipo de usuário é aceito do cliente sem aprovação

2. Permissões

O formulário cadastra paciente, mas o backend permite criar medico com qualquer CRM não vazio. PUT aceita tipo_usuario sem a lista de valores aplicada no cadastro. A manipulação do atributo é comprovada; nenhum privilégio médico adicional foi encontrado no código atual, portanto não se afirma escalada a administrador.

Impacto

Falsificação do tipo de conta e da identidade profissional registrada; inconsistência de papéis. A tomada de conta pelo PUT é contada separadamente em A02.

Condições de explorabilidade

POST exige e-mail ainda não cadastrado, campos obrigatórios e CRM não vazio para medico. PUT exige campos obrigatórios, mas não autorização. Um esquema de banco com restrição de domínio pode recusar valores fora da lista; medico continua permitido.

Arquivo por arquivo, linha por linha

[../insulog-mobile-front-clone/lib/services/api/auth_service.dart:89-95](#)

```
89:     await _apiService.post('usuarios', {
90:       'nome': fullName,
91:       'email': email,
92:       'senha': password,
93:       'tipo_login': 'email',
94:       'tipo_usuario': 'paciente',
95:       'id_medico': 16
```

[src/services/userService.js:48-58](#)

```
48:   const tipoNormalizado = tipo_usuario.toLowerCase()
49:
50:   if (!['medico', 'paciente'].includes(tipoNormalizado)) {
51:     const error = new Error('tipo_usuario invalido')
52:     error.statusCode = 400
53:     throw error
54:   }
55:
56:   if (tipoNormalizado === 'medico' && !crm) {
57:     const error = new Error('crm e obrigatorio para usuarios do tipo medico')
58:     error.statusCode = 400
```

[src/services/userService.js:70-74](#)

```
70:   const senhaCriptografada = passwordService.hashPassword(senha)
71:
72:   return await userRepository.create(
73:     { nome, email, senha: senhaCriptografada, tipo_login, tipo_usuario, id_medico, crm },
74:     tipoNormalizado
```

A06 · Evidências e análise · continuação

src/services/userService.js:78-91

```
78: async function updateUser(id, data) {
79:   const { nome, email, senha, tipo_login, tipo_usuario, id_medico, crm } = data
80:
81:   if (!nome || !email || !senha || !tipo_login || !tipo_usuario) {
82:     const error = new Error('Todos os campos sao obrigatorios')
83:     error.statusCode = 400
84:     throw error
85:   }
86:
87:   const senhaCriptografada = passwordService.hashPassword(senha)
88:
89:   return await userRepository.update(
90:     id,
91:     { nome, email, senha: senhaCriptografada, tipo_login, tipo_usuario, id_medico, crm }
```

src/repositories/userRepository.js:97-100

```
97:   if (tipoNormalizado === 'medico') {
98:     await conn.execute(
99:       `INSERT INTO medico (id_usuario, crm) VALUES (?, ?)`,
100:      [idUserario, crm]
```

src/repositories/userRepository.js:134-136

```
134:   await conn.execute(
135:     'UPDATE usuario SET nome = ?, email = ?, senha = ?, tipo_login = ?, tipo_usuario = ? WHERE i
    d_usuario = ?',
136:     [nome, email, senha, tipo_login, tipo_usuario, id]
```

Correção recomendada

Fixar o tipo permitido no cadastro público e criar processo autorizado de validação/aprovação para profissional, conforme regra do produto. Remover tipo_usuario do DTO de autoedição e validar domínio de valores no backend.

A07 · Evidências e análise

ALTA

Credenciais MySQL permanecem recuperáveis no Git

4. Chaves expostas

O arquivo `.env` foi versionado com `DB_USER=root` e `DB_PASSWORD` em texto claro. Foram identificados dois valores distintos em blobs históricos, apesar de o arquivo não existir no HEAD. O commit inicial já continha uma credencial; uma revisão posterior contém a segunda. Valores foram ocultados neste relatório para evitar nova distribuição.

Impacto

Leitores do histórico obtêm credenciais de banco. Se ainda válidas ou reutilizadas em serviço alcançável, permitem acesso com os privilégios configurados para essa conta. Não foram testadas contra nenhum serviço.

Condições de explorabilidade

Exposição histórica confirmada; uso atual, validade e alcance de rede não aferidos. `DB_HOST` era localhost, o que limita acesso direto remoto e não elimina risco de reutilização ou acesso ao host. Severidade alta condicionada à credencial ainda utilizável.

Arquivo por arquivo, linha por linha

`.env:1-5 @ 147015e9442d`

```
1: DB_HOST=localhost
2: DB_PORT=3306
3: DB_USER=root
4: DB_PASSWORD=[VALOR OCULTADO]
5: DB_NAME=insulog
```

`.env:1-5 @ 9ec3abca65b`

```
1: DB_HOST=localhost
2: DB_PORT=3306
3: DB_USER=root
4: DB_PASSWORD=[VALOR OCULTADO]
5: DB_NAME=insulog
```

`src/config/database.js:8-13`

```
8: const pool = mysql.createPool({
9:   host: process.env.DB_HOST,
10:  port: Number(process.env.DB_PORT),
11:  user: process.env.DB_USER,
12:  password: process.env.DB_PASSWORD,
13:  database: process.env.DB_NAME,
```

`.gitignore:1-5`

```
1: node_modules/
2: .env
3: .env.local
4: .env.development
5: .env.production
```

A07 · Evidências e análise · continuação

Correção recomendada

Rotacionar as duas credenciais expostas e qualquer reutilização; adotar conta de aplicação com privilégio mínimo. Só depois tratar histórico e cópias sob controle da equipe, com coordenação dos clones. Manter segredos fora do Git e adicionar detector no CI.

A08 · Evidências e análise

ALTA

Senhas e respostas sensíveis são gravadas nos logs

Adicional

O middleware registra req.body integralmente antes dos handlers, incluindo password do login e senha do cadastro/edição. Cadastro duplica o log no controller e no serviço. O interceptor também registra respostas completas, incluindo hashes da listagem e dados clínicos.

Impacto

Operadores ou terceiros com acesso aos logs obtêm senhas utilizáveis e dados pessoais/clínicos. A reprodução com valores fictícios confirmou senha e hash nos logs.

Condições de explorabilidade

Requer requisição com dados sensíveis e acesso posterior à saída de logs; não depende de modo debug ou feature flag. Não foi verificada a retenção ou ACL de um provedor de logs.

Arquivo por arquivo, linha por linha

src/app.js:39-48

```
39:     const retorno = Buffer.isBuffer(body)
40:       ? body.toString('utf8')
41:       : body
42:
43:     console.log('<retorno>', {
44:       metodo: req.method,
45:       rota: req.originalUrl,
46:       status: res.statusCode,
47:       body: retorno
48:     })
```

src/app.js:51-58

```
51:   res.json = function jsonInterceptado(body) {
52:     registrarRetorno(body)
53:     return originalJson(body)
54:   }
55:
56:   res.send = function sendInterceptado(body) {
57:     registrarRetorno(body)
58:     return originalSend(body)
```

src/app.js:78-85

```
78: app.use((req, res, next) => {
79:   if (Object.keys(req.query).length > 0) {
80:     console.log('Query:', req.query)
81:   }
82:
83:   if (req.body && Object.keys(req.body).length > 0) {
84:     console.log('Body:', req.body)
85:   }
```


A08 · Evidências e análise · continuação

[src/controllers/userController.js:43-46](#)

```
43: async function create(req, res, next) {  
44:   try {  
45:     console.log('Request body:', req.body)  
46:     const user = await userService.createUser(req.body)
```

[src/services/userService.js:37-40](#)

```
37: async function createUser(data) {  
38:   const { nome, email, senha, tipo_login, tipo_usuario, id_medico, crm } = data  
39:  
40:   console.log('createUser:', data)
```

Correção recomendada

Substituir dumps de body/resposta por metadados permitidos e redigir campos sensíveis recursivamente quando necessário. Retirar logs duplicados e tratar acesso/retenção dos logs existentes; avaliar troca das senhas expostas.

A09 · Evidências e análise

MÉDIA

App persiste senha em preferências locais

Adicional

Após login bem-sucedido, o app grava a senha original em SharedPreferences e a recupera para novo login automático. O código não utiliza armazenamento protegido para esse valor. O bundle local também contém a gravação da chave saved_password, sem uma senha literal embutida.

Impacto

Quem obtiver acesso ao armazenamento da aplicação pode recuperar a senha reutilizável. No artefato web, as preferências usam armazenamento do navegador; não foi demonstrado um XSS para acessá-lo.

Condições de explorabilidade

Login bem-sucedido e acesso ao armazenamento local, perfil do navegador ou contexto comprometido. Não se afirma que um outro app Android sem privilégios possa ler o sandbox.

Arquivo por arquivo, linha por linha

[../insulog-mobile-front-clone/lib/services/local/saved_login_service.dart:20-28](#)

```
20: Future<void> saveCredentials({
21:   required int userId,
22:   required String username,
23:   required String password,
24: }) async {
25:   final prefs = await SharedPreferences.getInstance();
26:   await prefs.setInt(_userIdKey, userId);
27:   await prefs.setString(_usernameKey, username);
28:   await prefs.setString(_passwordKey, password);
```

[../insulog-mobile-front-clone/lib/services/local/saved_login_service.dart:31-35](#)

```
31: Future<SavedLoginData?> getCredentials() async {
32:   final prefs = await SharedPreferences.getInstance();
33:   final userId = prefs.getInt(_userIdKey);
34:   final username = prefs.getString(_usernameKey);
35:   final password = prefs.getString(_passwordKey);
```

[../insulog-mobile-front-clone/lib/states/login_form_state.dart:54-59](#)

```
54:   final loginData = await AuthService().login(username, password);
55:   await _savedLoginService.saveCredentials(
56:     userId: loginData.userId,
57:     username: username,
58:     password: password,
59:   );
```

[../insulog-mobile-front-clone/lib/states/login_form_state.dart:111-115](#)

```
111:   try {
112:     final loginData = await AuthService().login(
113:       savedCredentials.username,
114:       savedCredentials.password,
115:     );
```

A09 · Evidências e análise · continuação

Correção recomendada

Eliminar persistência de senha. Usar sessão revogável/expirável, com armazenamento apropriado à plataforma: Keychain/Keystore no app nativo e cookie HttpOnly/Secure quando a arquitetura web permitir. Limpar a chave antiga na migração.

A10 · Evidências e análise

ALTA

App envia login e dados clínicos por HTTP

Adicional

A URL base é construída exclusivamente com http://, e o manifesto Android principal permite tráfego sem TLS. A senha é enviada em JSON para /login por essa mesma URL. O relatório também usa a URL base sem HTTPS.

Impacto

Observador ou intermediário no caminho de rede pode ler credenciais e dados clínicos; um intermediário ativo pode alterar respostas. Isto independe de CORS e do hash da senha no banco.

Condições de explorabilidade

Atacante com visibilidade/controla o caminho de rede e uso da URL HTTP produzida pelo app. Um túnel externo pode mitigar o transporte, mas não foi encontrado no escopo. Não houve interceptação de tráfego real.

Arquivo por arquivo, linha por linha

[../insulog-mobile-front-clone/lib/services/local/api_ip_service.dart:18-26](#)

```
18: Future<String> getBaseUrl() async {
19:   final savedIp = await getApiIpDigits();
20:   final ip =
21:     (isValidIp(savedIp) ? savedIp : null) ??
22:     formatDigitsAsIp(savedIp) ??
23:     formatDigitsAsIp(_defaultApiIpDigits) ??
24:     '10.173.57.47';
25:
26:   return 'http://$ip:$_apiPort';
```

[../insulog-mobile-front-clone/lib/services/api/auth_service.dart:12-17](#)

```
12: Future<LoginData> login(String username, String password) async {
13:   try {
14:     final response = await _apiService.post('login', {
15:       'username': username,
16:       'password': password,
17:     });
```

[../insulog-mobile-front-clone/lib/services/api/api_service.dart:28-32](#)

```
28:   _logRequest('POST', uri, body: body);
29:
30:   final response = await http
31:     .post(uri, headers: headers, body: jsonEncode(body))
32:     .timeout(const Duration(seconds: 10));
```

[../insulog-mobile-front-clone/android/app/src/main/AndroidManifest.xml:10-15](#)

```
10: <application
11:   android:label="insulog"
12:   android:name="${applicationName}"
13:   android:icon="@mipmap/ic_launcher"
14:   android:enableOnBackInvokedCallback="true"
```

A10 · Evidências e análise · continuação

```
15:         android:usesCleartextTraffic="true">
```

[../insulog-mobile-front-clone/lib/services/api/report_export_service.dart:58-70](#)

```
58:         final uri = Uri.parse('${await _baseUrl()}/exportacoes/relatorio')
59:         .replace(
60:             queryParams: {
61:                 'id_usuario': '$userId',
62:                 'dataInicio': firstDay,
63:                 'dataFim': lastDay,
64:                 'formato': format.name,
65:             },
66:         );
67:         final get = _client?.get ?? http.get;
68:         final response = await get(
69:             uri,
70:             headers: {'Accept': format.mimeType},
```

Correção recomendada

Disponibilizar API com HTTPS e certificado válido, permitir URL segura no cliente e bloquear tráfego HTTP nas configurações de produção. Se HTTP for necessário no desenvolvimento, restringi-lo a configuração específica de debug.

Recomendações priorizadas

P1 — Restabelecer confiança no servidor

Implementar identidade autenticada e política de posse antes de expor dados reais; corrigir A01–A04 em conjunto. Restringir imediatamente acesso público enquanto a correção não estiver concluída.

P1 — Conter exposição de credenciais

Rotacionar as duas senhas históricas (A07), remover senhas de logs (A08) e verificar retenção/acesso. A decisão de reescrever histórico deve ser coordenada e ocorre após a rotação.

P2 — Autorizar manutenção e cadastro profissional

Definir política no backend para catálogos e tipo de conta (A05/A06); não depender do payload ou da UI.

P2 — Proteger transporte e sessão do app

HTTPS em release (A10), sessão revogável e migração de saved_password (A09). Adaptar o cliente ao mecanismo de autenticação implementado.

P3 — Prevenir regressões

Converter reproduções em testes negativos de 401/403/404 e estado preservado; cobrir anônimo, dono, outro usuário e profissional vinculado/não vinculado. Adicionar detecção de segredos ao CI.

Validação e limites da auditoria

- Severidades qualitativas pelo impacto e precondições; não são escores CVSS nem prova de comprometimento em produção.
- Histórico: 19 commits da API e 17 do app, ambos clones não rasos; 1.001 e 334 blobs únicos, incluindo dependências vendorizadas na API. Reflogs, objetos órfãos, forks e refs remotas não presentes não foram auditados.
- Os 49 registros de rota foram inspecionados. Reproduções fizeram 49 requisições: 48 rotas alcançáveis e uma checagem do sombreamento; SQL substituído em memória. Não validam constraints, permissões MySQL nem dados existentes no ambiente real.
- A suíte original passou em 21 testes usando dependências isoladas em /tmp, após a instalação local original falhar por ausência de exceljs. Nenhum código de aplicação foi alterado.
- Bundle web apareceu durante a auditoria e foi inspecionado em leitura; não foi compilado pelo auditor nem atestado como saída do HEAD. 40 arquivos inventariados; 1 match de padrão era tabela codificada do runtime, sem segredo configurado. SHA-256 de main.dart.js:
a9bc4b23582622eee2bee7dc94a2e46b56d70eef839297127b86beab8474c03f.
- Busca de padrões de segredo, com triagem dos candidatos, não garante ausência de todo segredo. Valores históricos confirmados foram ocultados; não foi testada validade nem reuso.
- Arquivos de terceiros foram varridos para segredos, mas não receberam auditoria funcional de dependências. Arquivos binários e assets não tiveram revisão de conteúdo completa. Testes Flutter/ dispositivo não foram executados.

Artefatos reproduzíveis: verificar-auditoria.cjs; validacao-http.json; cobertura-rotas.csv; inventario-e-varredura.json; verificacao-bundle.json. O script de reprodução afirma o comportamento vulnerável atual; seus resultados positivos não significam que a aplicação esteja segura.

Cobertura de todos os handlers

49 registros Express: 48 alcançáveis e 1 sombreado. A matriz CSV contém o encadeamento completo até funções dos repositories. Abaixo, cada registro com resultado da revisão. As leituras de catálogos e o endpoint de saúde/login têm exposição intencional compatível com o código; demais rotas não têm identidade/posse validada.

Método / rota	Rota / controller	Resultado da revisão
GET /	src/app.js:90 inline:90-94	Público intencional; mensagem de saúde da API; sem dados de usuário.
GET /alarmes	src/routes/alarmeRoutes.js:6 src/controllers/alarmeController.js:3 (index)	Sem autenticação e sem autorização/posse; listagem global A01.
GET /alarmes/usuario/:usuariold	src/routes/alarmeRoutes.js:7 src/controllers/alarmeController.js:12 (showByUsuariold)	Sem autenticação e sem autorização/posse; ID do path/query/body controla dados A03.
GET /alarmes/:id	src/routes/alarmeRoutes.js:8 src/controllers/alarmeController.js:22 (show)	Sem autenticação e sem autorização/posse; ID do path/query/body controla dados A03.
POST /alarmes	src/routes/alarmeRoutes.js:9 src/controllers/alarmeController.js:32 (create)	Sem autenticação e sem autorização/posse; ID do path/query/body controla dados A03.
PUT /alarmes/:id	src/routes/alarmeRoutes.js:10 src/controllers/alarmeController.js:41 (update)	Sem autenticação e sem autorização/posse; ID do path/query/body controla dados A03.
DELETE /alarmes/:id	src/routes/alarmeRoutes.js:11 src/controllers/alarmeController.js:51 (deleteById)	Sem autenticação e sem autorização/posse; ID do path/query/body controla dados A03.
POST /login	src/routes/authRoutes.js:6 src/controllers/authController.js:3 (login)	Público intencional; verifica senha; não emite token/sessão.
GET /configuracoes	src/routes/configuracaoRoutes.js:6 src/controllers/configuracaoController.js:3 (index)	Sem autenticação e sem autorização/posse; listagem global A01.
GET /configuracoes/:id	src/routes/configuracaoRoutes.js:7 src/controllers/configuracaoController.js:12 (show)	Sem autenticação e sem autorização/posse; ID do path/query/body controla dados A03.
POST /configuracoes	src/routes/configuracaoRoutes.js:8 src/controllers/configuracaoController.js:22 (create)	Sem autenticação e sem autorização/posse; ID do path/query/body controla dados A03.
PUT /configuracoes/:id	src/routes/configuracaoRoutes.js:9 src/controllers/configuracaoController.js:31 (update)	Sem autenticação e sem autorização/posse; ID do path/query/body controla dados A03.

Cobertura de todos os handlers

Método / rota	Rota / controller	Resultado da revisão
DELETE /configuracoes/:id	src/routes/configuracaoRoutes.js:10 src/controllers/ configuracaoController.js:41 (deleteById)	Sem autenticação e sem autorização/ posse; ID do path/query/body controla dados A03.
GET /exportacoes	src/routes/exportacaoRoutes.js:6 src/controllers/exportacaoController.js:3 (index)	Sem autenticação e sem autorização/ posse; listagem global A01.
GET /exportacoes/relatorio	src/routes/exportacaoRoutes.js:7 src/controllers/ exportacaoController.js:51 (gerarRelatorio)	Sem autenticação e sem autorização/ posse; ID da query controla exportação A04.
GET /exportacoes/:id	src/routes/exportacaoRoutes.js:8 src/controllers/ exportacaoController.js:12 (show)	Sem autenticação e sem autorização/ posse; ID do path/query/body controla dados A03.
POST /exportacoes	src/routes/exportacaoRoutes.js:9 src/controllers/ exportacaoController.js:22 (create)	Sem autenticação e sem autorização/ posse; ID do path/query/body controla dados A03.
PUT /exportacoes/:id	src/routes/exportacaoRoutes.js:10 src/controllers/ exportacaoController.js:31 (update)	Sem autenticação e sem autorização/ posse; ID do path/query/body controla dados A03.
DELETE /exportacoes/:id	src/routes/exportacaoRoutes.js:11 src/controllers/ exportacaoController.js:41 (deleteById)	Sem autenticação e sem autorização/ posse; ID do path/query/body controla dados A03.
GET /periodos	src/routes/periodoRoutes.js:6 src/controllers/periodoController.js:3 (index)	Leitura de catálogo global; sem dado privado identificado.
GET /periodos/:id	src/routes/periodoRoutes.js:7 src/controllers/periodoController.js:12 (show)	Leitura de catálogo global; sem dado privado identificado.
POST /periodos	src/routes/periodoRoutes.js:8 src/controllers/periodoController.js:22 (create)	Sem autenticação e sem autorização/ posse; A05.
PUT /periodos/:id	src/routes/periodoRoutes.js:9 src/controllers/periodoController.js:31 (update)	Sem autenticação e sem autorização/ posse; A05.
DELETE /periodos/:id	src/routes/periodoRoutes.js:10 src/controllers/periodoController.js:41 (deleteById)	Sem autenticação e sem autorização/ posse; A05.

Cobertura de todos os handlers

Método / rota	Rota / controller	Resultado da revisão
GET /registros-glicose	src/routes/registroGlicoseRoutes.js:6 src/controllers/ registroGlicoseController.js:3 (index)	Sem autenticação e sem autorização/ posse; listagem global A01.
GET /registros-glicose/ dashboard	src/routes/registroGlicoseRoutes.js:7 src/controllers/ registroGlicoseController.js:44 (getDashboard)	Sem autenticação e sem autorização/ posse; ID do path/query/body controla dados A03.
GET /registros-glicose/ usuario/:id_usuario/historico	src/routes/registroGlicoseRoutes.js:8 src/controllers/ registroGlicoseController.js:54 (getHistorico)	Sem autenticação e sem autorização/ posse; ID do path/query/body controla dados A03.
GET /registros-glicose/ usuario/:id_usuario	src/routes/registroGlicoseRoutes.js:9 src/controllers/ registroGlicoseController.js:12 (showById)	Sem autenticação e sem autorização/ posse; ID do path/query/body controla dados A03.
GET /registros-glicose/:id	src/routes/registroGlicoseRoutes.js:10 src/controllers/ registroGlicoseController.js:34 (show)	Sem autenticação e sem autorização/ posse; ID do path/query/body controla dados A03.
POST /registros-glicose	src/routes/registroGlicoseRoutes.js:11 src/controllers/ registroGlicoseController.js:70 (create)	Sem autenticação e sem autorização/ posse; ID do path/query/body controla dados A03.
PUT /registros-glicose/:id	src/routes/registroGlicoseRoutes.js:12 src/controllers/ registroGlicoseController.js:79 (update)	Sem autenticação e sem autorização/ posse; ID do path/query/body controla dados A03.
DELETE /registros-glicose/:id	src/routes/registroGlicoseRoutes.js:13 src/controllers/ registroGlicoseController.js:89 (deleteById)	Sem autenticação e sem autorização/ posse; ID do path/query/body controla dados A03.
GET /registros-insulina	src/routes/registroInsulinaRoutes.js:6 src/controllers/ registroInsulinaController.js:3 (index)	Sem autenticação e sem autorização/ posse; listagem global A01.
GET /registros-insulina/ usuario/:id_usuario	src/routes/registroInsulinaRoutes.js:7 src/controllers/ registroInsulinaController.js:22 (showById)	Sem autenticação e sem autorização/ posse; ID do path/query/body controla dados A03.
GET /registros-insulina/:id	src/routes/registroInsulinaRoutes.js:8 src/controllers/ registroInsulinaController.js:12 (show)	Sem autenticação e sem autorização/ posse; ID do path/query/body controla dados A03.

Cobertura de todos os handlers

Método / rota	Rota / controller	Resultado da revisão
POST /registros-insulina	src/routes/registroInsulinaRoutes.js:9 src/controllers/ registroInsulinaController.js:33 (create)	Sem autenticação e sem autorização/ posse; ID do path/query/body controla dados A03.
PUT /registros-insulina/:id	src/routes/registroInsulinaRoutes.js:10 src/controllers/ registroInsulinaController.js:42 (update)	Sem autenticação e sem autorização/ posse; ID do path/query/body controla dados A03.
DELETE /registros-insulina/:id	src/routes/registroInsulinaRoutes.js:11 src/controllers/ registroInsulinaController.js:52 (deleteById)	Sem autenticação e sem autorização/ posse; ID do path/query/body controla dados A03.
GET /tipos-insulina	src/routes/typeInsuRoutes.js:6 src/controllers/typeInsuController.js:3 (index)	Leitura de catálogo global; sem dado privado identificado.
GET /tipos-insulina/:id	src/routes/typeInsuRoutes.js:7 src/controllers/typeInsuController.js:12 (show)	Leitura de catálogo global; sem dado privado identificado.
POST /tipos-insulina	src/routes/typeInsuRoutes.js:8 src/controllers/typeInsuController.js:22 (create)	Sem autenticação e sem autorização/ posse; A05.
PUT /tipos-insulina/:id	src/routes/typeInsuRoutes.js:9 src/controllers/typeInsuController.js:31 (update)	Sem autenticação e sem autorização/ posse; A05.
DELETE /tipos-insulina/:id	src/routes/typeInsuRoutes.js:10 src/controllers/typeInsuController.js:41 (deleteById)	Sem autenticação e sem autorização/ posse; A05.
GET /usuarios	src/routes/userRoutes.js:8 src/controllers/userController.js:3 (index)	Sem autenticação e sem autorização/ posse; A01.
GET /usuarios/:id	src/routes/userRoutes.js:9 src/controllers/userController.js:22 (show)	Sem autenticação e sem autorização/ posse; A02.
GET /usuarios/:tipo_usuario	src/routes/userRoutes.js:10 src/controllers/userController.js:12 (showByType)	Sombreado por GET /usuarios/:id; inspecionado, não alcançável no roteamento atual.
POST /usuarios	src/routes/userRoutes.js:12 src/controllers/userController.js:43 (create)	Sem autenticação e sem autorização/ posse; A06.
DELETE /usuarios/:id	src/routes/userRoutes.js:13 src/controllers/userController.js:33 (deleteById)	Sem autenticação e sem autorização/ posse; A02.

Cobertura de todos os handlers

Método / rota	Rota / controller	Resultado da revisão
PUT /usuarios/:id	src/routes/userRoutes.js:14 src/controllers/userController.js:53 (update)	Sem autenticação e sem autorização/ posse; A02/A06.

ISSUES PARA O GITHUB

Textos completos em Markdown, delimitados para cópia. Dez issues propostas; nenhuma foi publicada. Também disponíveis sem quebras de paginação em `issues-github.md`. A01-A04 devem compartilhar a implementação de autenticação, preservando testes específicos por superfície. Valores históricos permanecem ocultos.

--- ISSUE 1 — A01 ---

[Segurança] Listagens globais expõem dados e hashes de senha

Labels sugeridas: security, alta

Descrição

As seis listagens de dados privados não recebem identidade autenticada e executam consultas sem filtro de dono. GET /usuarios devolve SELECT * diretamente, incluindo a coluna senha. Os IDs expostos também facilitam as referências diretas dos demais achados.

Condições de explorabilidade

Acesso de rede à API e linhas existentes nas tabelas; não depende de login ou feature flag. Hashes não equivalem automaticamente a senhas recuperadas.

Evidência

src/controllers/userController.js:3-6

```
```text
```

```
3: async function index(req, res, next) {
4: try {
5: const users = await userService.listUsers()
6: return res.status(200).json(users)
7: }
8: }
```

src/repositories/userRepository.js:4-9

```
```text
```

```
4: async function findAll() {
5:   const [rows] = await db.execute(
6:     'SELECT * FROM usuario ORDER BY id_usuario ASC'
7:   )
8:   return rows
9: }
```

Demais locais afetados (trechos completos na análise A01 do relatório):

src/repositories/registroGlicoseRepository.js:5-9; src/repositories/registroInsulinaRepository.js:4-7; src/repositories/alarmeRepository.js:14-17; src/repositories/configuracaoRepository.js:4-7; src/repositories/exportacaoRepository.js:4-7; src/services/passwordService.js:42-47.

Impacto

Leitura em massa de contas, hashes scrypt, medições, doses, alarmes, preferências e metadados de exportação. Se houver contas legadas em texto puro, a senha também é retornada; a existência delas em produção não foi aferida.

ISSUES PARA O GITHUB · continuação

Sugestão de correção

Exigir identidade validada, filtrar cada coleção pelo dono autorizado ou vínculo médico-paciente verificado e retornar DTOs com lista explícita de campos. Nunca serializar senha/hash. Migrar senhas legadas caso existam.

Critérios de aceite

- [] Anônimo recebe 401 nas seis coleções privadas.
- [] Usuário A não recebe nenhuma linha de B, inclusive nas junções de insulina.
- [] Respostas de usuários nunca contêm senha, hash ou salt.
- [] Vínculo médico-paciente é validado no servidor antes de permitir leitura.

--- FIM ISSUE 1 ---

ISSUES PARA O GITHUB - 2

--- ISSUE 2 — A02 ---

[Segurança] Conta de terceiros pode ser alterada, inclusive a senha

Labels sugeridas: security, crítica

Descrição

GET, PUT e DELETE /usuarios/:id confiam apenas no ID. A atualização aceita uma nova senha, gera o hash e atualiza a conta sem sessão, senha atual ou fluxo de recuperação. A reprodução local mudou a senha de uma conta fictícia e realizou login com a nova senha.

Condições de explorabilidade

API acessível e ID existente. Para atualização, enviar os campos obrigatórios; nenhuma credencial prévia é exigida. DELETE pode ser impedido por integridade referencial, sem corrigir a falha de autorização.

Evidência

src/services/userService.js:78-91

```
```text
```

```
78: async function updateUser(id, data) {
79: const { nome, email, senha, tipo_login, tipo_usuario, id_medico, crm } = data
80:
81: if (!nome || !email || !senha || !tipo_login || !tipo_usuario) {
82: const error = new Error('Todos os campos sao obrigatorios')
83: error.statusCode = 400
84: throw error
85: }
86:
87: const senhaCriptografada = passwordService.hashPassword(senha)
88:
89: return await userRepository.update(
90: id,
91: { nome, email, senha: senhaCriptografada, tipo_login, tipo_usuario, id_medico, crm }
  ```
```

src/repositories/userRepository.js:134-136

```
```text
```

```
134: await conn.execute(
135: 'UPDATE usuario SET nome = ?, email = ?, senha = ?, tipo_login = ?, tipo_usuario = ? WHERE i
 d_usuario = ?',
136: [nome, email, senha, tipo_login, tipo_usuario, id]
  ```
```

Demais locais afetados (trechos completos na análise A02 do relatório):

src/routes/userRoutes.js:8-14; src/controllers/userController.js:53-57; src/repositories/userRepository.js:42-45; src/repositories/userRepository.js:18-30.

Impacto

ISSUES PARA O GITHUB · 2 · continuação

Tomada de conta e alteração de nome, e-mail e tipo de usuário; leitura de perfil. Exclusão também alcança SQL sem autorização, podendo depender das foreign keys e dos registros vinculados.

Sugestão de correção

Autenticar as rotas, vincular o perfil à identidade do servidor e exigir autorização específica para terceiros. Separar troca de senha de edição comum, com reautenticação ou token de recuperação de uso único; limitar exclusão.

Critérios de aceite

- [] Anônimo não consulta, altera nem exclui contas.
- [] Usuário A não altera nem remove B ao trocar o ID.
- [] Troca de senha exige prova adicional e revoga sessões conforme a política.
- [] Senha/e-mail de B permanecem intactos após tentativa não autorizada.

--- FIM ISSUE 2 ---

ISSUES PARA O GITHUB - 3

--- ISSUE 3 — A03 ---

[Segurança] Registros clínicos e objetos vinculados não validam posse

Labels sugeridas: security, crítica

Descrição

Os handlers de glicose, insulina, alarmes, configurações e metadados de exportação usam IDs de path/body/query sem comparar com dono autenticado. Verificar existência com findById não valida posse. POST aceita id_usuario ou id_registro de terceiros; PUT pode mudar esses vínculos. Dashboard e histórico também aceitam o paciente escolhido pelo chamador.

Condições de explorabilidade

API alcançável e IDs/referências válidos. Constraints podem rejeitar IDs inexistentes, mas não isolam donos existentes. Sem feature flag ou sessão necessária.

Evidência

src/services/registroGlicoseService.js:125-130

```
```text
```

```
125: function montarRegistroCompleto(data, registroAtual = {}) {
126: const id_usuario = data.id_usuario ?? registroAtual.id_usuario
127: const nivel_glicose = data.nivel_glicose ?? registroAtual.nivel_glicose
128: const id_periodo = data.id_periodo ?? registroAtual.id_periodo
129: const data_hora = data.data_hora ?? registroAtual.data_hora ?? formatarDataHoraAtual()
130: const observacao = data.observacao ?? registroAtual.observacao ?? null
 ...
```

src/repositories/registroGlicoseRepository.js:197-205

```
```text
```

```
197:   await conn.execute(
198:     'UPDATE registroglicose SET id_usuario = ?, nivel_glicose = ?, data_hora = ?, id_periodo = ?
    , observacao = ? WHERE id_registro = ?',
199:     [
200:       glicose.id_usuario,
201:       glicose.nivel_glicose,
202:       glicose.data_hora,
203:       glicose.id_periodo,
204:       glicose.observacao ?? null,
205:       id
  ...
```

src/repositories/registroInsulinaRepository.js:51-54

```
```text
```

```
51: const [result] = await conn.execute(
52: 'INSERT INTO registroinsulina (id_registro, id_tipo_insulina, unidade_insulina) VALUES (?, ?,
 ?)',
53: [id_registro, id_tipo_insulina, unidade_insulina]
54:)
```

## ISSUES PARA O GITHUB · 3 · continuação

...

Demais locais afetados (trechos completos na análise A03 do relatório):

src/controllers/registroGlicoseController.js:44-48; src/services/registroGlicoseService.js:356-365; src/repositories/registroGlicoseRepository.js:51-54; src/repositories/registroGlicoseRepository.js:277-290; src/repositories/registroGlicoseRepository.js:304-310; src/repositories/registroInsulinaRepository.js:12-15; src/repositories/registroInsulinaRepository.js:79-81; src/repositories/registroInsulinaRepository.js:100-104; src/repositories/alarmeRepository.js:22-25; src/repositories/alarmeRepository.js:30-33; src/repositories/alarmeRepository.js:46-48; src/repositories/alarmeRepository.js:79-81; src/repositories/alarmeRepository.js:105-108; src/repositories/configuracaoRepository.js:12-15; src/repositories/configuracaoRepository.js:28-30; src/repositories/configuracaoRepository.js:57-59; src/repositories/configuracaoRepository.js:79-82; src/repositories/exportacaoRepository.js:12-15; src/repositories/exportacaoRepository.js:28-30; src/repositories/exportacaoRepository.js:56-58; src/repositories/exportacaoRepository.js:77-80.

### ## Impacto

Leitura, criação, adulteração e exclusão de medições e doses de terceiros; alteração de alarmes e preferências. O impacto crítico decorre da integridade dos registros clínicos, não de um comando demonstrado a dispositivo de administração de insulina.

### ## Sugestão de correção

Exigir sessão e aplicar predicados de dono em leitura e escrita. Para insulina, validar o dono pela junção com registroglicose. Derivar id\_usuario da sessão, verificar referências cruzadas e impedir transferência arbitrária de dono; manter verificação e mutação na mesma transação quando necessário.

### ## Critérios de aceite

- [ ] Todos os handlers privados da matriz rejeitam anônimos.
- [ ] IDs de B em path, query e body de A são rejeitados sem alteração de estado.
- [ ] POST de insulina/alarme não permite referência a registro de outro paciente.
- [ ] Dashboard, histórico e buscas por usuário não revelam dados de B.
- [ ] Operações legítimas do dono continuam funcionando.

--- FIM ISSUE 3 ---

## ISSUES PARA O GITHUB · 4

### --- ISSUE 4 — A04 ---

#### # [Segurança] Relatório clínico pode ser exportado para qualquer paciente

Labels sugeridas: security, alta

#### ## Descrição

GET /exportacoes/relatorio recebe id\_usuario da query e repassa ao repository. A query filtra esse ID, mas nenhuma etapa verifica a identidade do solicitante ou o vínculo com o paciente. A validação de formato/data não é autorização.

#### ## Condições de explorabilidade

ID existente, intervalo válido e registros no período. Sem token/cookie necessário; download de PDF confirmado com banco simulado. Não foi acessado banco real.

#### ## Evidência

src/controllers/exportacaoController.js:54-60

```
```text
```

```
54:     const { id_usuario, dataInicio, dataFim, formato } = req.query
55:     const arquivo = await exportacaoService.gerarRelatorio({
56:       idUsuario: id_usuario,
57:       dataInicio,
58:       dataFim,
59:       formato
60:     })
```
```

src/repositories/exportacaoRepository.js:104-111

```
```text
```

```
104:     FROM registroglicose rg
105:     LEFT JOIN periodo p ON p.id_periodo = rg.id_periodo
106:     LEFT JOIN registroinsulina ri ON ri.id_registro = rg.id_registro
107:     LEFT JOIN tipoinsulina ti ON ti.id_tipo_insulina = ri.id_tipo_insulina
108:     WHERE rg.id_usuario = ? AND rg.data_hora >= ?
109:           AND rg.data_hora < DATE_ADD(?, INTERVAL 1 DAY)
110:     ORDER BY rg.data_hora ASC, ri.id_registro_insulina ASC`,
111:     [idUsuario, dataInicio, dataFim]
```
```

Demais locais afetados (trechos completos na análise A04 do relatório):

src/routes/exportacaoRoutes.js:6-9; src/services/exportacaoService.js:114-118; src/repositories/exportacaoRepository.js:84-87; ../insulog-mobile-front-clone/lib/services/api/report\_export\_service.dart:58-70.

#### ## Impacto

Extração de identidade e histórico clínico consolidado em PDF/XLSX de terceiros, com observações e aplicações de insulina.

## ISSUES PARA O GITHUB · 4 · continuação

### ## Sugestão de correção

Autenticar antes de gerar o relatório, usar o próprio ID da sessão ou validar explicitamente o vínculo com outro paciente. Reutilizar a mesma política para os dois formatos.

### ## Critérios de aceite

- [ ] Sem sessão: 401 para PDF e XLSX.
- [ ] Usuário A não exporta B alterando id\_usuario.
- [ ] Médico sem vínculo é rejeitado; vínculo autorizado é aceito.
- [ ] Cache-Control no-store e validação dos parâmetros continuam presentes.

--- FIM ISSUE 4 ---

# ISSUES PARA O GITHUB · 5

## --- ISSUE 5 — A05 ---

### # [Segurança] Catálogos compartilhados aceitam escrita anônima

Labels sugeridas: security, alta

#### ## Descrição

POST, PUT e DELETE de períodos e tipos de insulina não verificam privilégio. São catálogos globais sem dono individual; portanto a falha é de autorização funcional, e não IDOR de tenant. Não há gate de papel correspondente no frontend: os endpoints podem ser chamados diretamente.

#### ## Condições de explorabilidade

API acessível; campos descricao/nome preenchidos. PUT/DELETE exigem ID existente. Não foi presumido um papel admin que não existe no projeto.

#### ## Evidência

src/routes/periodoRoutes.js:8-10

```
```text
```

```
8: router.post('/', periodoController.create)
9: router.put('/:id', periodoController.update)
10: router.delete('/:id', periodoController.deleteById)
```
```

src/routes/typeInsuRoutes.js:8-10

```
```text
```

```
8: router.post('/', typeInsuController.create)
9: router.put('/:id', typeInsuController.update)
10: router.delete('/:id', typeInsuController.deleteById)
```
```

src/repositories/periodoRepository.js:54-56

```
```text
```

```
54:     await conn.execute(
55:         'UPDATE periodo SET descricao = ? WHERE id_perodo = ?',
56:         [descricao, id]
```
```

Demais locais afetados (trechos completos na análise A05 do relatório):

src/services/periodoService.js:40-46; src/repositories/periodoRepository.js:28-30; src/repositories/periodoRepository.js:73-76; src/repositories/typeInsuRepository.js:21-24; src/repositories/typeInsuRepository.js:36-38; src/repositories/typeInsuRepository.js:63-65.

#### ## Impacto

Qualquer chamador pode inserir/alterar rótulos compartilhados usados nos registros e relatórios. Exclusões de entradas vinculadas podem ser bloqueadas por foreign keys; inserção e edição continuam sem autorização.

## ISSUES PARA O GITHUB · 5 · continuação

### ## Sugestão de correção

Definir política de manutenção dos catálogos e aplicá-la no servidor às seis operações de escrita. Se não forem gerenciáveis pela API, remover as rotas de mutação.

### ## Critérios de aceite

- [ ] As seis operações rejeitam anônimo e usuário sem privilégio.
- [ ] Somente principal explicitamente autorizado altera catálogos.
- [ ] Testes cobrem POST, PUT e DELETE dos dois routers.

--- FIM ISSUE 5 ---

# ISSUES PARA O GITHUB - 6

## --- ISSUE 6 — A06 ---

### # [Segurança] Tipo de usuário é aceito do cliente sem aprovação

Labels sugeridas: security, média

#### ## Descrição

O formulário cadastra paciente, mas o backend permite criar medico com qualquer CRM não vazio. PUT aceita tipo\_usuario sem a lista de valores aplicada no cadastro. A manipulação do atributo é comprovada; nenhum privilégio médico adicional foi encontrado no código atual, portanto não se afirma escalada a administrador.

#### ## Condições de explorabilidade

POST exige e-mail ainda não cadastrado, campos obrigatórios e CRM não vazio para medico. PUT exige campos obrigatórios, mas não autorização. Um esquema de banco com restrição de domínio pode recusar valores fora da lista; medico continua permitido.

#### ## Evidência

../insulog-mobile-front-clone/lib/services/api/auth\_service.dart:89-95

```
```text
```

```
89:      await _apiService.post('usuarios', {
90:        'nome': fullName,
91:        'email': email,
92:        'senha': password,
93:        'tipo_login': 'email',
94:        'tipo_usuario': 'paciente',
95:        'id_medico': 16
96:      });
```

src/services/userService.js:48-58

```
```text
```

```
48: const tipoNormalizado = tipo_usuario.toLowerCase()
49:
50: if (!['medico', 'paciente'].includes(tipoNormalizado)) {
51: const error = new Error('tipo_usuario invalido')
52: error.statusCode = 400
53: throw error
54: }
55:
56: if (tipoNormalizado === 'medico' && !crm) {
57: const error = new Error('crm e obrigatorio para usuarios do tipo medico')
58: error.statusCode = 400
59: }
```

src/repositories/userRepository.js:134-136

```
```text
```

```
134:   await conn.execute(
135:     'UPDATE usuario SET nome = ?, email = ?, senha = ?, tipo_login = ?, tipo_usuario = ? WHERE i
    d_usuario = ?',
```

ISSUES PARA O GITHUB · 6 · continuação

```
136:      [nome, email, senha, tipo_login, tipo_usuario, id]
    ````
```

Demais locais afetados (trechos completos na análise A06 do relatório):

src/services/userService.js:70-74; src/services/userService.js:78-91; src/repositories/userRepository.js:97-100.

### ## Impacto

Falsificação do tipo de conta e da identidade profissional registrada; inconsistência de papéis. A tomada de conta pelo PUT é contada separadamente em A02.

### ## Sugestão de correção

Fixar o tipo permitido no cadastro público e criar processo autorizado de validação/aprovação para profissional, conforme regra do produto. Remover tipo\_usuario do DTO de autoedição e validar domínio de valores no backend.

### ## Critérios de aceite

- [ ] Cadastro público não atribui papel profissional sem o processo definido.
- [ ] Autoedição não altera tipo\_usuario.
- [ ] Tipo inválido é rejeitado no servidor.
- [ ] Testes documentam a política de cadastro de profissionais.

--- FIM ISSUE 6 ---



# ISSUES PARA O GITHUB · 7

## --- ISSUE 7 — A07 ---

### # [Segurança] Credenciais MySQL permanecem recuperáveis no Git

Labels sugeridas: security, alta

#### ## Descrição

O arquivo .env foi versionado com DB\_USER=root e DB\_PASSWORD em texto claro. Foram identificados dois valores distintos em blobs históricos, apesar de o arquivo não existir no HEAD. O commit inicial já continha uma credencial; uma revisão posterior contém a segunda. Valores foram ocultados neste relatório para evitar nova distribuição.

#### ## Condições de explorabilidade

Exposição histórica confirmada; uso atual, validade e alcance de rede não aferidos. DB\_HOST era localhost, o que limita acesso direto remoto e não elimina risco de reutilização ou acesso ao host. Severidade alta condicionada à credencial ainda utilizável.

#### ## Evidência

.env:1-5 @ 147015e9442d

```
```text
```

```
1: DB_HOST=localhost
2: DB_PORT=3306
3: DB_USER=root
4: DB_PASSWORD=[VALOR OCULTADO]
5: DB_NAME=insulog
```
```

.env:1-5 @ 9ec3abca65b

```
```text
```

```
1: DB_HOST=localhost
2: DB_PORT=3306
3: DB_USER=root
4: DB_PASSWORD=[VALOR OCULTADO]
5: DB_NAME=insulog
```
```

Demais locais afetados (trechos completos na análise A07 do relatório):

src/config/database.js:8-13; .gitignore:1-5.

#### ## Impacto

Leitores do histórico obtêm credenciais de banco. Se ainda válidas ou reutilizadas em serviço alcançável, permitem acesso com os privilégios configurados para essa conta. Não foram testadas contra nenhum serviço.

#### ## Sugestão de correção

Rotacionar as duas credenciais expostas e qualquer reutilização; adotar conta de aplicação com privilégio mínimo. Só depois tratar histórico e cópias sob controle da equipe, com coordenação dos clones. Manter segredos fora do Git e adicionar detector no CI.

## ISSUES PARA O GITHUB · 7 · continuação

### ## Critérios de aceite

- [ ] As duas credenciais antigas foram invalidadas, com confirmação do responsável pelo banco.
- [ ] Aplicação usa segredo externo e conta com privilégio mínimo.
- [ ] Varredura de segredos cobre commits no CI.
- [ ] Plano de tratamento do histórico/cópias foi executado ou documentado; remoção de HEAD não é aceita como rotação.

--- FIM ISSUE 7 ---

## ISSUES PARA O GITHUB · 8

### --- ISSUE 8 — A08 ---

#### # [Segurança] Senhas e respostas sensíveis são gravadas nos logs

Labels sugeridas: security, alta

#### ## Descrição

O middleware registra req.body integralmente antes dos handlers, incluindo password do login e senha do cadastro/edição. Cadastro duplica o log no controller e no serviço. O interceptor também registra respostas completas, incluindo hashes da listagem e dados clínicos.

#### ## Condições de explorabilidade

Requer requisição com dados sensíveis e acesso posterior à saída de logs; não depende de modo debug ou feature flag. Não foi verificada a retenção ou ACL de um provedor de logs.

#### ## Evidência

src/app.js:39-48

```
```text
```

```
39:   const retorno = Buffer.isBuffer(body)
40:     ? body.toString('utf8')
41:     : body
42:
43:   console.log('<retorno>', {
44:     metodo: req.method,
45:     rota: req.originalUrl,
46:     status: res.statusCode,
47:     body: retorno
48:   })
```
```

src/app.js:78-85

```
```text
```

```
78: app.use((req, res, next) => {
79:   if (Object.keys(req.query).length > 0) {
80:     console.log('Query:', req.query)
81:   }
82:
83:   if (req.body && Object.keys(req.body).length > 0) {
84:     console.log('Body:', req.body)
85:   }
```
```

Demais locais afetados (trechos completos na análise A08 do relatório):

src/app.js:51-58; src/controllers/userController.js:43-46; src/services/userService.js:37-40.

#### ## Impacto

Operadores ou terceiros com acesso aos logs obtêm senhas utilizáveis e dados pessoais/clínicos. A reprodução com valores fictícios confirmou senha e hash nos logs.

## ISSUES PARA O GITHUB · 8 · continuação

### ## Sugestão de correção

Substituir dumps de body/resposta por metadados permitidos e redigir campos sensíveis recursivamente quando necessário. Retirar logs duplicados e tratar acesso/retenção dos logs existentes; avaliar troca das senhas expostas.

### ## Critérios de aceite

- [ ] Login, cadastro e edição não deixam senha/password/hash nos logs.
- [ ] Respostas clínicas não são registradas integralmente.
- [ ] Testes capturam saída de log e validam ausência de valores sentinela.
- [ ] Logs preexistentes têm acesso e retenção revisados.

--- FIM ISSUE 8 ---

## ISSUES PARA O GITHUB · 9

### --- ISSUE 9 — A09 ---

#### # [Segurança] App persiste senha em preferências locais

Labels sugeridas: security, média

#### ## Descrição

Após login bem-sucedido, o app grava a senha original em SharedPreferences e a recupera para novo login automático. O código não utiliza armazenamento protegido para esse valor. O bundle local também contém a gravação da chave saved\_password, sem uma senha literal embutida.

#### ## Condições de explorabilidade

Login bem-sucedido e acesso ao armazenamento local, perfil do navegador ou contexto comprometido. Não se afirma que um outro app Android sem privilégios possa ler o sandbox.

#### ## Evidência

../insulog-mobile-front-clone/lib/services/local/saved\_login\_service.dart:20-28

```
```text
```

```
20: Future<void> saveCredentials({
21:   required int userId,
22:   required String username,
23:   required String password,
24: }) async {
25:   final prefs = await SharedPreferences.getInstance();
26:   await prefs.setInt(_userIdKey, userId);
27:   await prefs.setString(_usernameKey, username);
28:   await prefs.setString(_passwordKey, password);
29: }
```

../insulog-mobile-front-clone/lib/states/login_form_state.dart:54-59

```
```text
```

```
54: final loginData = await AuthService().login(username, password);
55: await _savedLoginService.saveCredentials(
56: userId: loginData.userId,
57: username: username,
58: password: password,
59:);
```

Demais locais afetados (trechos completos na análise A09 do relatório):

../insulog-mobile-front-clone/lib/services/local/saved\_login\_service.dart:31-35; ../insulog-mobile-front-clone/lib/states/login\_form\_state.dart:111-115.

#### ## Impacto

Quem obtiver acesso ao armazenamento da aplicação pode recuperar a senha reutilizável. No artefato web, as preferências usam armazenamento do navegador; não foi demonstrado um XSS para acessá-lo.

## ISSUES PARA O GITHUB · 9 · continuação

### ## Sugestão de correção

Eliminar persistência de senha. Usar sessão revogável/expirável, com armazenamento apropriado à plataforma: Keychain/Keystore no app nativo e cookie HttpOnly/Secure quando a arquitetura web permitir. Limpar a chave antiga na migração.

### ## Critérios de aceite

- [ ] Não existe senha original nas preferências após login.
- [ ] A migração remove saved\_password de instalações existentes.
- [ ] Logout/revogação invalidam a sessão e limpam o armazenamento.
- [ ] Autologin funciona com credencial de sessão revogável, sem guardar senha.

--- FIM ISSUE 9 ---

# ISSUES PARA O GITHUB · 10

## --- ISSUE 10 — A10 ---

### # [Segurança] App envia login e dados clínicos por HTTP

Labels sugeridas: security, alta

#### ## Descrição

A URL base é construída exclusivamente com http://, e o manifesto Android principal permite tráfego sem TLS. A senha é enviada em JSON para /login por essa mesma URL. O relatório também usa a URL base sem HTTPS.

#### ## Condições de explorabilidade

Atacante com visibilidade/controle do caminho de rede e uso da URL HTTP produzida pelo app. Um túnel externo pode mitigar o transporte, mas não foi encontrado no escopo. Não houve interceptação de tráfego real.

#### ## Evidência

../insulog-mobile-front-clone/lib/services/local/api\_ip\_service.dart:18-26

```
```text
```

```
18: Future<String> getBaseUrl() async {
19:   final savedIp = await getApiIpDigits();
20:   final ip =
21:     (isValidIp(savedIp) ? savedIp : null) ??
22:     formatDigitsAsIp(savedIp) ??
23:     formatDigitsAsIp(_defaultApiIpDigits) ??
24:     '10.173.57.47';
25:
26:   return 'http://$ip:$_apiPort';
```
```

../insulog-mobile-front-clone/lib/services/api/auth\_service.dart:12-17

```
```text
```

```
12: Future<LoginData> login(String username, String password) async {
13:   try {
14:     final response = await _apiService.post('login', {
15:       'username': username,
16:       'password': password,
17:     });
```
```

../insulog-mobile-front-clone/android/app/src/main/AndroidManifest.xml:10-15

```
```text
```

```
10: <application
11:   android:label="insulog"
12:   android:name="${applicationName}"
13:   android:icon="@mipmap/ic_launcher"
14:   android:enableOnBackInvokedCallback="true"
15:   android:usesCleartextTraffic="true">
```
```

## ISSUES PARA O GITHUB · 10 · continuação

Demais locais afetados (trechos completos na análise A10 do relatório):

../insulog-mobile-front-clone/lib/services/api/api\_service.dart:28-32; ../insulog-mobile-front-clone/lib/services/api/report\_export\_service.dart:58-70.

### ## Impacto

Observador ou intermediário no caminho de rede pode ler credenciais e dados clínicos; um intermediário ativo pode alterar respostas. Isto independe de CORS e do hash da senha no banco.

### ## Sugestão de correção

Disponibilizar API com HTTPS e certificado válido, permitir URL segura no cliente e bloquear tráfego HTTP nas configurações de produção. Se HTTP for necessário no desenvolvimento, restringi-lo a configuração específica de debug.

### ## Critérios de aceite

- [ ] Build de produção só envia credenciais e registros por HTTPS.
- [ ] HTTP e certificado inválido são rejeitados em produção.
- [ ] Manifesto de release não permite tráfego em claro.
- [ ] Download PDF/XLSX usa a mesma política TLS.

--- FIM ISSUE 10 ---